

[Dashboard](#) / [My courses](#) / [ITB IF1210 2 2526](#) / [Praktikum 7](#) / [ADT Stack & Queue - Praktikum \(STI\)](#)

Started on	Thursday, 30 April 2026, 12:54 AM
State	Finished
Completed on	Thursday, 30 April 2026, 2:37 AM
Time taken	1 hour 42 mins
Grade	250.00 out of 300.00 (83%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Sistem Antrean Wahana Nangor Falls

Nama File: antrean_wahana.c

Deskripsi Masalah

Paman Stun baru saja membuka wahana baru di Mystery Shack: **"Portal Dimensional Simulator"** — sebuah atraksi yang mengklaim bisa membawa pengunjung merasakan sensasi menembus dimensi lain (padahal cuma ruangan gelap dengan proyektor hologram murah). Seperti biasa, Paman Stun punya ide cemerlang untuk memaksimalkan profit:

Paman Stun: "Dengar, anak-anak! Aku punya strategi bisnis baru. Kita jual dua jenis tiket: *Regular* seharga \$5 dan *Priority Pass* seharga \$20! Yang bayar mahal masuk duluan, yang pelit... ya nunggu aja lah!"

Sistem ini langsung menimbulkan masalah. Pengunjung dengan tiket regular protes karena selalu didahulukan oleh pemegang Priority Pass. Bahkan ada yang sudah menunggu 30 menit tapi belum juga terpanggil karena pemegang Priority Pass terus berdatangan! Sementara itu, beberapa pengunjung yang kesal memilih pergi meninggalkan antrean karena kehabisan kesabaran.

Mebel: "Paman Stun! Orang-orang mulai komplain! Yang tiket regular pada kabur semua!"

Deeper: "Ini masalah klasik *starvation* di sistem antrean. Kita butuh mekanisme *fairness* dan juga harus handle yang udah kehabisan sabar..."

Paman Stun: "Gak ngerti! Yang penting kamu bikin sistem yang tetap ngasih prioritas ke yang bayar mahal tapi yang regular juga gak diterlantarkan. Dan kalau ada yang kabur... ya sudahlah, artinya mereka gak layak masuk Mystery Shack!"

Spesifikasi Sistem

Tugas kamu adalah membantu Deeper dan Mebel mengimplementasikan sistem manajemen antrean wahana yang **adil namun tetap memberi keuntungan pada pemegang Priority Pass**. Sistem ini harus:

- Mengelola **dua antrean terpisah**: antrean prioritas dan antrean regular
- Menerapkan **starvation prevention**: setelah melayani 3 pengunjung prioritas berturut-turut, sistem harus melayani 1 pengunjung regular (jika ada) sebelum kembali melayani prioritas
- Menangani **pengunjung yang kehabisan kesabaran**: setiap pengunjung punya batas waktu tunggu (*patience*), dan mereka akan pergi jika sudah menunggu terlalu lama
- Mencatat **waktu kedatangan** setiap pengunjung untuk menghitung berapa lama mereka sudah menunggu

File Header yang Disediakan

Unduh file-file header berikut:

- [pengunjung.h](#) — Struktur data pengunjung (ID, waktu kedatangan, kesabaran)
- [queue.h](#) — ADT Queue untuk menyimpan pengunjung
- [antrean_wahana.h](#) — Sistem manajemen antrean wahana (yang akan kamu implementasikan)
- [boolean.h](#) — Tipe boolean

File Implementasi

File [queue.c](#) sudah disediakan lengkap. Kamu hanya perlu mengimplementasikan file `antrean_wahana.c` yang berisi logika bisnis sistem antrean wahana.

Program Utama (Driver)

Buatlah file program utama (driver) Anda sendiri untuk menguji implementasi dari `antrean_wahana.c`. Anda bisa menggunakan [main.c](#) sebagai referensi pengetesan.

Catatan: Saat pengumpulan ke sistem grader, pastikan file `antrean_wahana.c` **tidak berisi** fungsi `main()`.

C

 [antrean_wahana.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	12	Accepted	0.00 sec, 1.71 MB
2	12	Accepted	0.00 sec, 1.67 MB
3	12	Accepted	0.00 sec, 1.64 MB
4	12	Accepted	0.00 sec, 1.65 MB
5	12	Accepted	0.00 sec, 1.71 MB
6	12	Accepted	0.00 sec, 1.67 MB
7	12	Accepted	0.00 sec, 1.71 MB
8	16	Accepted	0.00 sec, 1.67 MB

Question **2**

Partially correct

Mark 50.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: gravity.c

Anomali Gravitasi

Jauh di dalam hutan Pinus Gravity, kembar Dipper Pinus dan Mebel Pinus telah menemukan anomali gravitasi aneh yang melayang di udara. Setiap anomali memiliki ukuran (nilai absolut) dan arah gerakan (positif untuk bergerak ke kanan menuju Shack Misteri, negatif untuk bergerak ke kiri menuju hutan).

Dipper, peneliti yang penuh rasa ingin tahu, telah mendokumentasikan bahwa ketika dua anomali bertemu berhadapan, hal berikut terjadi:

- Jika satu anomali lebih besar dari yang lain, anomali yang lebih kecil akan hilang.
- Jika kedua anomali memiliki ukuran yang sama, keduanya saling menghancurkan sepenuhnya.
- Jika kedua anomali bergerak dalam arah yang sama, mereka tidak akan pernah bertemu dan terus berjalan.

Mebel (yang bersikeras dipanggil Mebel, bukan "Mabel") bertanya kepada Dipper: "Apa yang akan tersisa setelah semua anomali gravitasi ini saling bertabrakan?"

Dipper membutuhkan bantuan Anda untuk mensimulasikan keadaan akhir dari anomali-anomali ini setelah semua tabrakan terjadi.

Format Masukan:

n
a1 a2 a3 ... an

- n: jumlah anomali gravitasi ($2 \leq n \leq 10,000$)
- ai: ukuran dan arah anomali (negatif = bergerak ke kiri, positif = bergerak ke kanan; $-1000 \leq a_i \leq 1000$, $a_i \neq 0$)

Format Keluaran:

Sisa anomali gravitasi setelah semua tabrakan

- Cetak anomali yang tersisa dalam urutan asli mereka muncul dalam ruang-waktu Gravity Pinus
- Jika tidak ada anomali tersisa, cetak "Celah telah tertutup"

Contoh Masukan dan Keluaran:

No	Masukan	Keluaran	Keterangan
1.	4 2 4 -4 -1	2	Anomali 4 dan -4 bertabrakan dan saling menghancurkan (ukuran sama). Anomali -1 bertabrakan dengan anomali 4 yang tersisa dan hilang (ukuran lebih kecil). Hanya anomali 2 yang tersisa bergerak ke kanan.
2.	2 5 5	5 5	Kedua anomali bergerak ke arah yang sama (positif), sehingga tidak akan pernah bertabrakan. Keduanya terus berjalan.
3.	3 7 -3 9	7 9	Anomali 7 bergerak ke kanan. Anomali -3 bertabrakan dengan 9, tetapi 9 lebih besar sehingga -3 hilang. Anomali 7 dan 9 tidak pernah bertabrakan karena bergerak menjauh. Hasil: 7 dan 9.

Catatan:

- Setiap baris output harus diakhiri dengan newline ("
")
- **Wajib** untuk menggunakan stack yang telah terdefinisi dan diimplementasikan pada prapraktikum untuk soal ini.
- File ADT tidak perlu dikumpulkan, hanya driver terkait yang dikumpulkan.

C



Score: 50

Blackbox

Score: 50

Verdict: Wrong answer

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.00 sec, 1.58 MB
2	0	Wrong answer	0.00 sec, 1.60 MB
3	0	Wrong answer	0.00 sec, 1.71 MB
4	10	Accepted	0.00 sec, 1.50 MB
5	10	Accepted	0.00 sec, 1.50 MB
6	10	Accepted	0.00 sec, 1.54 MB
7	0	Wrong answer	0.00 sec, 1.61 MB
8	10	Accepted	0.00 sec, 1.52 MB
9	0	Wrong answer	0.00 sec, 1.62 MB
10	0	Wrong answer	0.00 sec, 1.57 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: minqueue.c

Catatan: Implementasikan ADT yang telah didefinisikan melalui file [minqueue.h](#) untuk soal ini!

Antrian Prioritas Anomali Pinus Gravity

Dipper Pinus dan Mebel Pinus telah menemukan bahwa anomali gravitasi di Pinus Gravity dapat dikategorikan berdasarkan tingkat ancaman mereka. Untuk mengelola anomali ini secara efisien, mereka membutuhkan sebuah sistem antrian khusus yang dapat:

- **Menambah anomali** ke belakang antrian (enqueue)
- **Memproses anomali** dari depan antrian (dequeue)
- **Mengetahui anomali paling berbahaya** (anomali dengan tingkat ancaman terendah) dalam antrian saat ini secara $O(1)$

MinQueue adalah struktur data antrian khusus yang dapat menemukan elemen minimum dalam waktu $O(1)$. Struktur ini menggunakan dua pasang stack (inbox/outbox dengan masing-masing stack minimum tracking) untuk mencapai efisiensi ini.

Dipper membutuhkan bantuan Anda untuk mengimplementasikan sistem manajemen anomali ini agar mereka dapat menangani anomali dengan prioritas terendah (tingkat ancaman terendah) terlebih dahulu.

Format Masukan:

n
cmd1
cmd2
...
cmdn

- n: jumlah operasi yang akan dilakukan ($1 \leq n \leq 100$)
- cmd: perintah operasi MinQueue, dengan format berikut:
 - **E x**: Enqueue (tambah) anomali dengan tingkat ancaman x ($-1000 \leq x \leq 1000$)
 - **D**: Dequeue (proses) anomali dari depan antrian
 - **M**: Dapatkan tingkat ancaman minimum dalam antrian saat ini
 - **C**: Check apakah antrian kosong (output 1 jika kosong, 0 jika tidak kosong)

Format Keluaran:

Untuk setiap operasi, cetak hasilnya sesuai dengan tabel di bawah:

- **E x**: cetak "OK" jika berhasil ditambahkan, atau "PENUH" jika antrian penuh
- **D**: cetak nilai anomali yang diproses, atau "KOSONG" jika antrian kosong
- **M**: cetak tingkat ancaman minimum, atau "KOSONG" jika antrian kosong
- **C**: cetak "1" jika kosong, "0" jika tidak kosong

Contoh Masukan dan Keluaran:

No	Masukan	Keluaran	Keterangan
1.	5	OK	Tambah 10 (OK), tambah 5 (OK), minimum adalah 5, tambah 3 (OK), minimum sekarang adalah 3.
	E 10	OK	
	E 5	5	
	M	OK	
	E 3	3	
	M		
2.	6	OK	Tambah 7, 2, 9 (semua OK). Dequeue: hasil 7 (depan antrian). Minimum dalam antrian saat ini: 2. Minimum tetap 2.
	E 7	OK	
	E 2	OK	
	E 9	7	
	D	2	
	M	2	
	M		

3.	4 C E 42 C M	1 OK 0 42	Check kosong: 1 (kosong). Tambah 42 (OK). Check kosong: 0 (tidak kosong). Minimum: 42.
----	--------------------------	--------------------	--

Catatan:

- Setiap baris output harus diakhiri dengan newline ("\\n")
- MinQueue memiliki kapasitas maksimal 100 elemen (CAPACITY = 100)
- Tingkat ancaman dapat berupa angka negatif

C

 [minqueue.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	10	Accepted	0.00 sec, 1.59 MB
2	10	Accepted	0.00 sec, 1.66 MB
3	10	Accepted	0.00 sec, 1.66 MB
4	10	Accepted	0.00 sec, 1.59 MB
5	10	Accepted	0.00 sec, 1.65 MB
6	10	Accepted	0.00 sec, 1.65 MB
7	10	Accepted	0.00 sec, 1.55 MB
8	10	Accepted	0.00 sec, 1.62 MB
9	10	Accepted	0.00 sec, 1.60 MB
10	10	Accepted	0.00 sec, 1.62 MB

[◀ ADT Stack & Queue - Pra Praktikum \(STI\)](#)

Jump to...

[ADT Stack & Queue - Latihan Praktikum \(STI\) ▶](#)